FLUTTER COURSE · SESSION 7 OF 15

Local Storage + Checkpoint 1

Every app so far forgets everything when it closes. Today you'll save small things on the phone (settings, favourites) so they're still there next time, and combine that with Riverpod. Then you start your first checkpoint project.

BY THE END OF TODAY YOU CAN

Save and load simple values and lists with `shared_preferences`, load saved data when the app starts, and keep saved data in sync with Riverpod state.

Setup

```
flutter pub add shared_preferences
```

Today's apps are in the **Session 7** group of the Run and Debug dropdown. It works on Android, iOS, desktop and Chrome; in Chrome the data is kept in the browser.

HOW TO TEST SAVING

After saving something, **fully restart** the app: stop it and run again, or refresh the Chrome tab. Hot reload keeps everything in memory, so it doesn't prove anything was saved.

In Chrome, always use the same port, or a new run will start with empty storage: `flutter run -d chrome --web-port 5050`.

The big picture: a shelf of labelled jars

Normal variables live in the app's short-term memory. Close the app and they're gone. `shared_preferences` is a **shelf of labelled jars** on the phone. Each jar has a **key** (its label) and holds one small thing.

"name"	"darkMode"	"openCount"	"favouriteCities"
"Ravi"	true	4	["Pune","Delhi"]
String	bool	int	List<String>

The shelf is for **small** things: settings, favourites, a login token. A big collection, like all your notes, belongs in a database. That's Firestore, in Session 9.

Storage words, in plain English

CODE	WHAT IT DOES
`await SharedPreferences.getInstance()`	Open the shelf. Do this once, then keep the object.
`prefs.setString('name', 'Ravi')`	Put a value in the jar labelled `name`. Also `setBool`, `setInt`, `setDouble`, `setStringList`.
`prefs.getString('name')`	Look in the jar. Returns **null** if it was never filled. Also `getBool`, `getInt`, `getDouble`, `getStringList`.
`prefs.getInt('openCount') ?? 0`	Use 0 when nothing was saved yet (`??` from Session 1).
`prefs.remove('name')`	Empty one jar.
`prefs.clear()`	Empty every jar.
`jsonEncode(quote.toJson())`	The shelf only holds simple types. To save an object, turn it into a JSON String first.
`Quote.fromJson(jsonDecode(text))`	...and turn it back into an object when you load it.
`WidgetsFlutterBinding.ensureInitialized()`	Required in an `async main()` before using a plugin like shared_preferences.
`overrideWithValue(prefs)`	Plug the opened shelf into a provider when the app starts, so every screen can use it with no waiting.

The concepts, in the order we cover them

`demo1` Saving simple values

A settings screen: your name, a dark-mode switch, and how many times you've opened the app. Change them, restart, and they're still there.

```
const _kName = 'name';   // keys as constants: a typo can't create a second jar

// Load when the screen starts (called from initState)
final prefs = await SharedPreferences.getInstance();
_nameController.text = prefs.getString(_kName) ?? '';
_darkMode = prefs.getBool(_kDarkMode) ?? false;

// Save
await prefs.setString(_kName, _nameController.text.trim());
await prefs.setBool(_kDarkMode, value);
```

While the shelf is opening, the screen shows a spinner. Opening is fast, but it's still a `Future`.

`demo2` Saving a list

Tap hearts on a list of cities. The names of your favourites are saved as a `List<String>`. Restart and the hearts are still red.

```
_favourites = prefs.getStringList('favouriteCities') ?? [];   // load

await prefs.setStringList('favouriteCities', _favourites);    // save the whole list after every change
```

`demo3` Storage + Riverpod

The same cities, but the favourites live in a Notifier. It **loads** them in `build()` and **saves** them every time they change. The screens just call `toggle()`; they don't know storage exists.

```
// 1. A provider for the shelf, with no real value yet
final sharedPreferencesProvider = Provider<SharedPreferences>((ref) {
  throw UnimplementedError('Override sharedPreferencesProvider in main()');
});

// 2. main opens the shelf BEFORE the app starts, and plugs it in
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final prefs = await SharedPreferences.getInstance();
  runApp(ProviderScope(
    overrides: [sharedPreferencesProvider.overrideWithValue(prefs)],
    child: const MyApp(),
  ));
}

// 3. The Notifier loads at the start and saves on every change
class FavouritesNotifier extends Notifier<List<String>> {
  @override
  List<String> build() =>
      ref.watch(sharedPreferencesProvider).getStringList('favouriteCities') ?? [];   // load

  void toggle(String city) {
    state = state.contains(city) ? state.where((c) => c != city).toList() : [...state, city];
    ref.read(sharedPreferencesProvider).setStringList('favouriteCities', state);    // save
  }
}
```

Because `main` opens the shelf first, there's no loading spinner anywhere, and every screen sees the saved favourites on the very first frame.

The project: Quotes App

Sessions 6 and 7 together. The main screen loads 20 quotes from the internet. Tap the heart to save a quote to **My favourites**. Favourites are saved on the device, so they're still there after a restart, and they work with **no internet**. Pull down to load a different set of quotes.

```
lib/quotes_app/
├─ main.dart · async main, opens storage, overrideWithValue
├─ routes.dart · class AppRoutes
├─ models/quote.dart · fromJson AND toJson
├─ services/quote_service.dart · fetches quotes, error messages
├─ providers/quote_providers.dart · quotesProvider + FavouritesNotifier
├─ widgets/quote_card.dart · used on both screens
└─ screens/ · quotes_screen.dart, favourites_screen.dart
```

FILE	WHAT TO LOOK FOR
`quote.dart`	`toJson` uses the same keys as the API, so `fromJson` reads both the API and the saved copies.
`quote_providers.dart`	Favourites are saved as a list of JSON strings, one per quote. Saving the whole quote, not just its id, is what makes them work offline.
`quotes_screen.dart`	`.when`, pull to refresh, and an error screen with a shortcut to your saved favourites.
`favourites_screen.dart`	Reads only from storage. Works with no internet. Has an empty state.
`test/quotes_app_test.dart`	Checks that a favourite is saved, and that saved favourites load on start and work offline.

Errors you will see today

> **Saved data is gone after a restart**
>
> You forgot `await` on `set...`, or you tested with hot reload instead of a full restart.

> **`In Chrome, everything is gone after you stop and run the app again`**
>
> Flutter picks a new random port on every run (`localhost:61207`, then `localhost:54321` …), and Chrome keeps saved data separately for each address. Run on a fixed port: `flutter run -d chrome --web-port 5050` (the Session 7 entries in VS Code already do this). Refreshing the tab keeps the same port, so it always works. Phones and emulators don't have this problem.

> **`"Binding has not yet been initialized"`**
>
> Add `WidgetsFlutterBinding.ensureInitialized();` as the first line of your `async main()`.

> **`"UnimplementedError: Override sharedPreferencesProvider in main()"`**
>
> You forgot `overrides: [sharedPreferencesProvider.overrideWithValue(prefs)]` in `ProviderScope`.

> **`You can't save a model object`**
>
> The shelf only holds simple types. Use `jsonEncode(item.toJson())` to save and `Item.fromJson(jsonDecode(text))` to load.

> **`Old test data keeps showing in Chrome`**
>
> Chrome keeps the data for each address. Clear it with `prefs.clear()` (demo 1 has a "Forget everything" button).

Practice task (in class)

- [ ] Open your Session 6 practice app (the posts list).
- [ ] Add a heart to each post that marks it as a favourite.
- [ ] Save the favourites with `shared_preferences`. They're still red after a full restart.
- [ ] Add a Favourites screen with an empty state.

CHECKPOINT PROJECT 1 · FUNDAMENTALS CHECK · SESSIONS 1–7

Product Catalog App

Build a shopping catalog from scratch. Products come from a real API, you can open each one, mark favourites, and your favourites are still there after a restart. This checks everything we've covered: Dart, widgets, layouts, navigation, forms, Riverpod, APIs and local storage.

What the app does

- **Product list:** loads products and shows them in a grid or list: thumbnail, title, price, rating.
- **Product detail:** tap a product to see a large image, title, brand, price, discount, rating, stock and description, plus a favourite button.
- **Favourites:** a screen listing your favourite products, saved on the device and still there after a restart.
- **Search:** a validated form field (at least 2 characters) that searches products.
- Proper **loading**, **empty** and **error** states on every screen that loads data, with Retry.

API (free, no key, works in Chrome)

ADDRESS	GIVES YOU
`https://dummyjson.com/products?limit=30`	`{"products": [...], "total": 194, ...}`. The list is inside `"products"`.
`https://dummyjson.com/products/1`	One product
`https://dummyjson.com/products/search?q=phone`	Search results, same shape as the list
`https://dummyjson.com/products/category-list`	Category names (bonus)

Model

```
class Product {
  final int id;
  final String title;
  final String description;
  final String category;
  final double price;              // prices are in US dollars, show "$"
  final double discountPercentage;
  final double rating;
  final int stock;
  final String? brand;             // some products have NO brand → nullable
  final String thumbnail;
  final List<String> images;
```

```
  factory Product.fromJson(Map<String, dynamic> json) { ... }
  Map<String, dynamic> toJson() { ... }   // for saving favourites
}
```

TWO TRAPS IN THIS API

Numbers: a price of 10 arrives as `10`, not `10.0`, so `json['price'] as double` crashes. Write `(json['price'] as num).toDouble()` for `price`, `discountPercentage` and `rating`.

Missing brand: groceries have no `brand` key. Use `json['brand'] as String?` and show nothing (or "No brand") when it's null.

Rules

1. **Named routes** with an `AppRoutes` class. Pass the product (or its id) to the detail screen.
2. **Typed model classes** with `fromJson`. No `json['...']` in widgets.
3. **All network code in a service class**, with a timeout and readable messages for no internet, server errors and bad data.
4. **Riverpod:** a `FutureProvider` for the products, and a `NotifierProvider` for favourites.
5. **Favourites persist** with `shared_preferences`: they're still there after a full restart, and the Favourites screen works offline.
6. **Four states** (loading, error with Retry, empty, data) on the list, search and detail screens.
7. **Theme only:** colours and fonts from `ThemeData`, no `Colors.xxx` in screens.
8. **Form validation:** the search field rejects empty input and fewer than 2 characters.
9. **No overflow** on a 360 px-wide phone. Long titles use `TextOverflow.ellipsis`.
10. **Git:** at least 6 commits with clear messages, pushed to GitHub, with a README and screenshots.

Grading

AREA	WHAT WE CHECK	POINTS
Dart & models	fromJson/toJson, null safety (brand), num → double, clean classes	15

Widgets & layout	Grid or list, detail layout, theme only, no overflow	15
Navigation & forms	Named routes, data passed to detail, validated search	15
Riverpod	FutureProvider, favourites Notifier, watch vs read used correctly	15
API integration	Service class, timeout, readable errors, Retry works	15
Local storage	Favourites persist across restarts and work offline	15
Code quality & Git	Folders (models, services, providers, screens, widgets), commits, README	10
Total		**100**

Bonus

- Category chips from `/products/category-list` that filter the list (`/products/category/{name}`).
- An image carousel on the detail screen using `images` and a `PageView`.
- Pull to refresh on the list.

How to submit

- [ ] Push to a public GitHub repository.
- [ ] README with screenshots of the list, detail, favourites, one error state and one empty state.
- [ ] Run `flutter analyze` and fix every warning.
- [ ] Send the repository link to your teacher.

Check yourself

▼ **Why is a normal variable lost when the app restarts?**

It lives in the app's memory, which is cleared when the app closes.
shared_preferences writes to the device's storage instead.

▼ **What does prefs.getString('name') return if nothing was saved?**

null. That's why we write `prefs.getString('name') ?? ''`.

▼ How do you save a whole object?

Turn it into a JSON String with `jsonEncode(object.toJson())`, and back with `fromJson(jsonDecode(text))`.

▼ Why call ensureInitialized() before getInstance() in main?

Plugins need Flutter's engine to be ready. In an async main, nothing has started it yet, so we start it ourselves.

▼ What belongs in shared_preferences, and what doesn't?

Small things: settings, favourites, a token. Not large collections of data. Those go in a database such as Firestore.

NEXT SESSION

Session 8 · Firebase Setup & Authentication. Real accounts: register, log in, log out, and stay logged in after a restart. The Paws & Homes login screen gets a real backend.